Tên: Lê Thiên Hạo

MSSV: 089206010393

1. Nạp dự liệu iris vào bộ nhớ và hiển thị thông tin về số thuộc tính (dimensionality), số objects (size).

In [1]:
```
from sklearn.datasets import load_iris

iris = load_iris(as_frame= True)
```

In [2]:
```
iris
```

Out[2]:
```
{'data':      sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
 0                   5.1               3.5                1.4
0.2
 1                   4.9               3.0                1.4
0.2
 2                   4.7               3.2                1.3
0.2
 3                   4.6               3.1                1.5
0.2
 4                   5.0               3.6                1.4
0.2
 ..                  ...               ...                ...
...
 145                 6.7               3.0                5.2
2.3
 146                 6.3               2.5                5.0
1.9
 147                 6.5               3.0                5.2
2.0
 148                 6.2               3.4                5.4
2.3
 149                 5.9               3.0                5.1
1.8

 [150 rows x 4 columns],
 'target': 0      0
 1      0
 2      0
 3      0
 4      0
       ..
 145    2
 146    2
 147    2
 148    2
 149    2
 Name: target, Length: 150, dtype: int64,
 'frame':      sepal length (cm)  sepal width (cm)  petal length (cm)  peta
l width (cm)  \
 0                   5.1               3.5                1.4
0.2
 1                   4.9               3.0                1.4
0.2
 2                   4.7               3.2                1.3
0.2
 3                   4.6               3.1                1.5
0.2
 4                   5.0               3.6                1.4
0.2
 ..                  ...               ...                ...
...
 145                 6.7               3.0                5.2
2.3
 146                 6.3               2.5                5.0
1.9
```

```
 147                6.5               3.0                5.2
2.0
 148                6.2               3.4                5.4
2.3
 149                5.9               3.0                5.1
1.8

      target
 0         0
 1         0
 2         0
 3         0
 4         0
 ..      ...
 145       2
 146       2
 147       2
 148       2
 149       2

 [150 rows x 5 columns],
 'target_names': array(['setosa', 'versicolor', 'virginica'], dtype='<U1
0'),
 'DESCR': '.. _iris_dataset:\n\nIris plants dataset\n--------------------\n
\n**Data Set Characteristics:**\n\n:Number of Instances: 150 (50 in each of
three classes)\n:Number of Attributes: 4 numeric, predictive attributes and
the class\n:Attribute Information:\n    - sepal length in cm\n    - sepal w
idth in cm\n    - petal length in cm\n    - petal width in cm\n    - clas
s:\n            - Iris-Setosa\n            - Iris-Versicolour\n
- Iris-Virginica\n\n:Summary Statistics:\n\n============== ==== ==== ======
= ===== ====================\n                Min  Max   Mean    SD   Class
Correlation\n============== ==== ==== ======= ===== ====================\ns
epal length:   4.3  7.9   5.84   0.83    0.7826\nsepal width:    2.0  4.4
3.05   0.43   -0.4194\npetal length:   1.0  6.9   3.76   1.76    0.9490  (h
igh!)\npetal width:    0.1  2.5   1.20   0.76    0.9565  (high!)\n=========
===== ==== ==== ======= ===== ====================\n\n:Missing Attribute Va
lues: None\n:Class Distribution: 33.3% for each of 3 classes.\n:Creator: R.
A. Fisher\n:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\n:Date:
July, 1988\n\nThe famous Iris database, first used by Sir R.A. Fisher. The
dataset is taken\nfrom Fisher\'s paper. Note that it\'s the same as in R, b
ut not as in the UCI\nMachine Learning Repository, which has two wrong data
points.\n\nThis is perhaps the best known database to be found in the\npatt
ern recognition literature.  Fisher\'s paper is a classic in the field and
\nis referenced frequently to this day.  (See Duda & Hart, for example.)  T
he\ndata set contains 3 classes of 50 instances each, where each class refe
rs to a\ntype of iris plant.  One class is linearly separable from the othe
r 2; the\nlatter are NOT linearly separable from each other.\n\n.. dropdow
n:: References\n\n  - Fisher, R.A. "The use of multiple measurements in tax
onomic problems"\n    Annual Eugenics, 7, Part II, 179-188 (1936); also in
"Contributions to\n    Mathematical Statistics" (John Wiley, NY, 1950).\n
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysi
s.\n    (Q327.D83) John Wiley & Sons.  ISBN 0-471-22361-1.  See page 218.\n
- Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System\n
Structure and Classification Rule for Recognition in Partially Exposed\n
Environments".  IEEE Transactions on Pattern Analysis and Machine\n    Inte
lligence, Vol. PAMI-2, No. 1, 67-71.\n  - Gates, G.W. (1972) "The Reduced N
```

```
earest Neighbor Rule".  IEEE Transactions\n    on Information Theory, May 1
972, 431-433.\n  - See also: 1988 MLC Proceedings, 54-64.  Cheeseman et a
l"s AUTOCLASS II\n    conceptual clustering system finds 3 classes in the d
ata.\n  - Many, many more ...\n',
 'feature_names': ['sepal length (cm)',
  'sepal width (cm)',
  'petal length (cm)',
  'petal width (cm)'],
 'filename': 'iris.csv',
 'data_module': 'sklearn.datasets.data'}
```

In [3]:
```
iris.data.head(20)
```

Out[3]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
5	5.4	3.9	1.7	0.4
6	4.6	3.4	1.4	0.3
7	5.0	3.4	1.5	0.2
8	4.4	2.9	1.4	0.2
9	4.9	3.1	1.5	0.1
10	5.4	3.7	1.5	0.2
11	4.8	3.4	1.6	0.2
12	4.8	3.0	1.4	0.1
13	4.3	3.0	1.1	0.1
14	5.8	4.0	1.2	0.2
15	5.7	4.4	1.5	0.4
16	5.4	3.9	1.3	0.4
17	5.1	3.5	1.4	0.3
18	5.7	3.8	1.7	0.3
19	5.1	3.8	1.5	0.3

Hiển thị thông tin của thuộc tính

In [4]:
```
iris.data.shape
```

Out[4]:
```
(150, 4)
```

In [5]:
```
iris.data.columns
```

Out[5]:
```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)'],
      dtype='object')
```

2. Đặt nhãn (label) cho dữ liệu đã nạp và hiển thị số lớp (classes)

In [6]:
```
labels = ['setosa', 'versicolor', 'virginica']
```

In [7]:
```
iris.target = iris.target.apply(lambda x: labels[x])
```

In [8]:
```
iris.target
```

Out[8]:
```
0         setosa
1         setosa
2         setosa
3         setosa
4         setosa
          ...
145    virginica
146    virginica
147    virginica
148    virginica
149    virginica
Name: target, Length: 150, dtype: object
```

3. Tách dữ liệu thành hai phần: training data(70%) và test data (30%)

In [9]:
```
from sklearn.model_selection import train_test_split
train_x , test_x, train_y, test_y  = train_test_split(iris.data, iris.target
```

4. Chuẩn hóa dữ liệu với StandardScaler

In [10]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
fitted_train_x = scaler.fit_transform(train_x)
fitted_test_x = scaler.fit_transform(test_x)
```

5. Tạo và huấn luyện mô hình với training data

In [11]:
```
from sklearn.tree import DecisionTreeClassifier

tree_clf = DecisionTreeClassifier(max_depth=2)
tree_clf.fit(fitted_train_x, train_y)
```

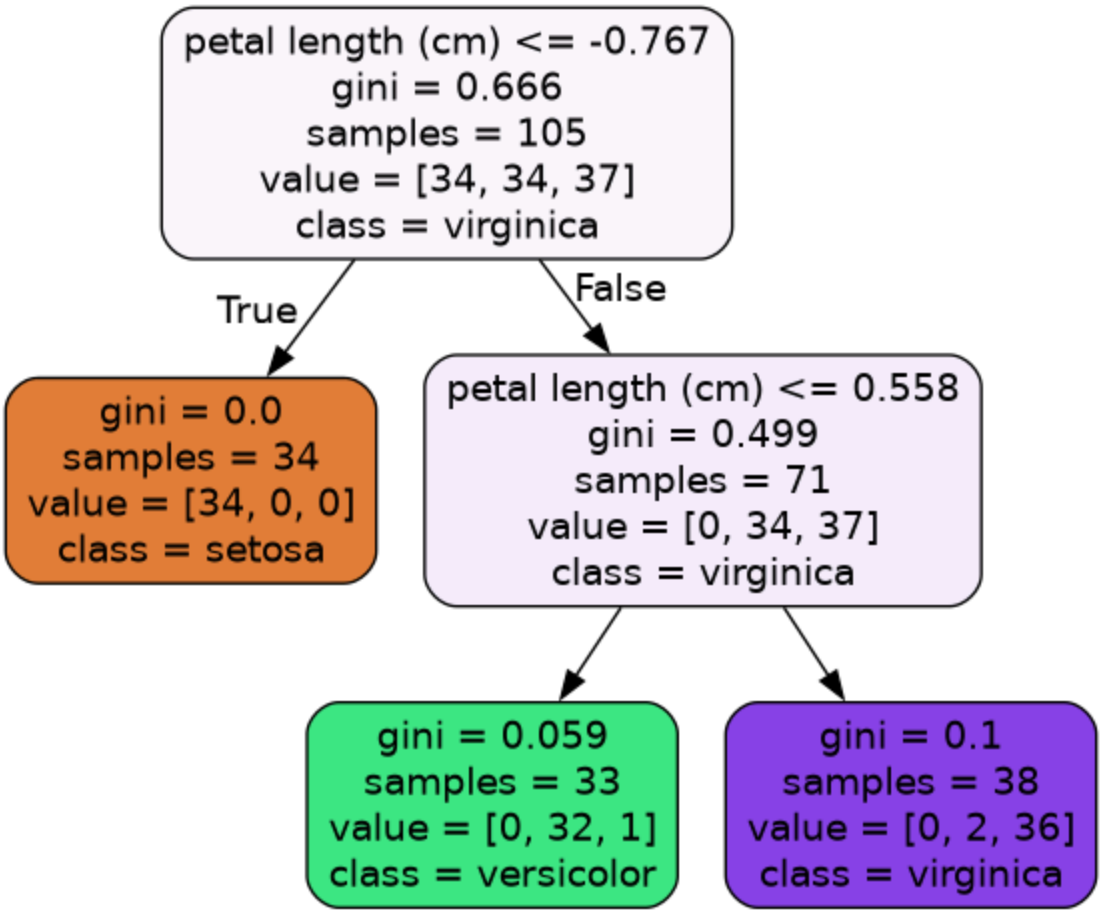
Out[11]:

▾ DecisionTreeClassifier

▶ Parameters

In [12]:
```
from sklearn.tree import export_graphviz
export_graphviz(
    tree_clf,
    out_file="iris.dot",
    feature_names=train_x.columns,
    class_names=labels,
    rounded=True,
    filled=True
)
```

In [13]:
```
!dot -Tpng iris.dot -o iris.png
```



6. Test và đánh giá độ chính xác của mô mình, yêu cầu hiển thị ma trận confusion matrix, accuracy, f-score

In [14]:
```
pred = tree_clf.predict(fitted_test_x)
```

In [ ]:
```
```

In [15]:
```
from sklearn.metrics import accuracy_score, precision_score, f1_score, class
```

In [16]:
```
print(classification_report(test_y, pred))
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        17
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.89      0.94        18

    accuracy                           0.96        45
   macro avg       0.94      0.96      0.95        45
weighted avg       0.96      0.96      0.96        45
```

In [17]:
```
confusion_matrix(test_y,pred)
```

Out[17]:
```
array([[17,  0,  0],
       [ 0, 10,  0],
       [ 0,  2, 16]])
```

In [18]:
```
from sklearn.metrics import ConfusionMatrixDisplay
import numpy as np
import matplotlib.pyplot as plt
```

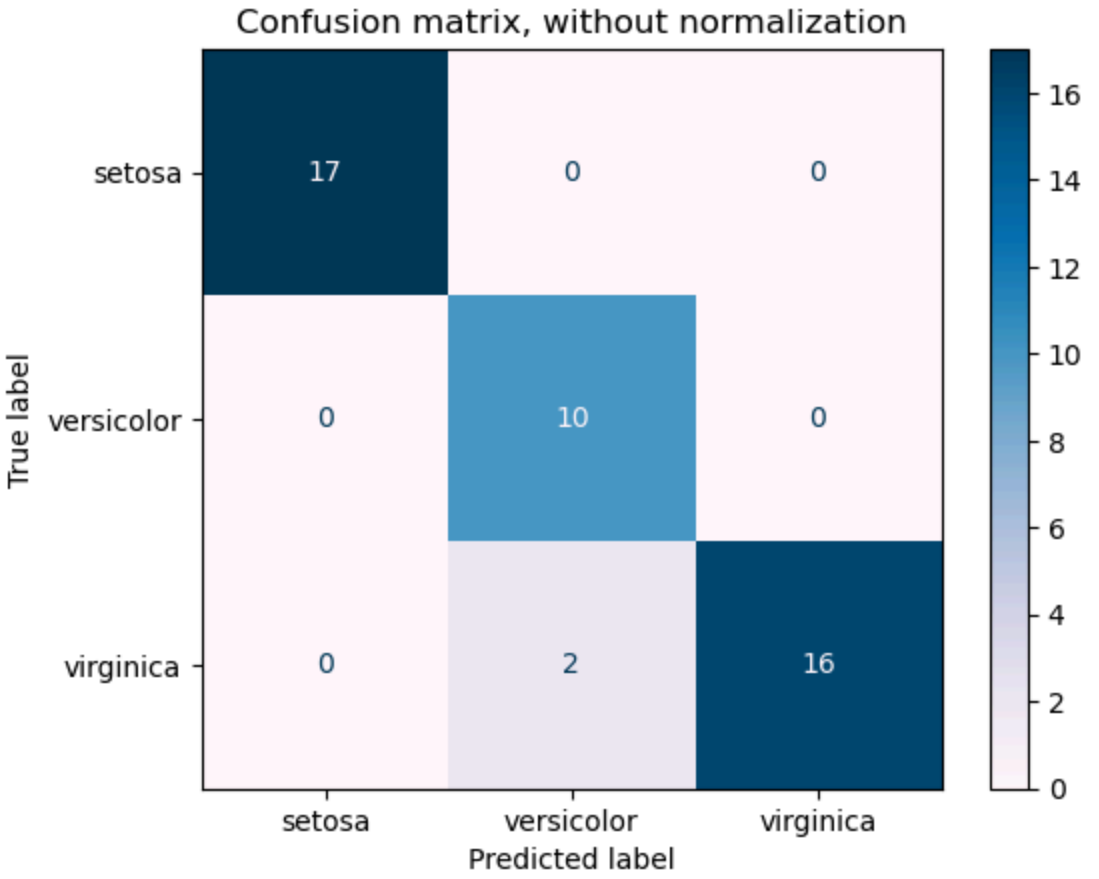
In [19]:
```
np.set_printoptions(precision=2)

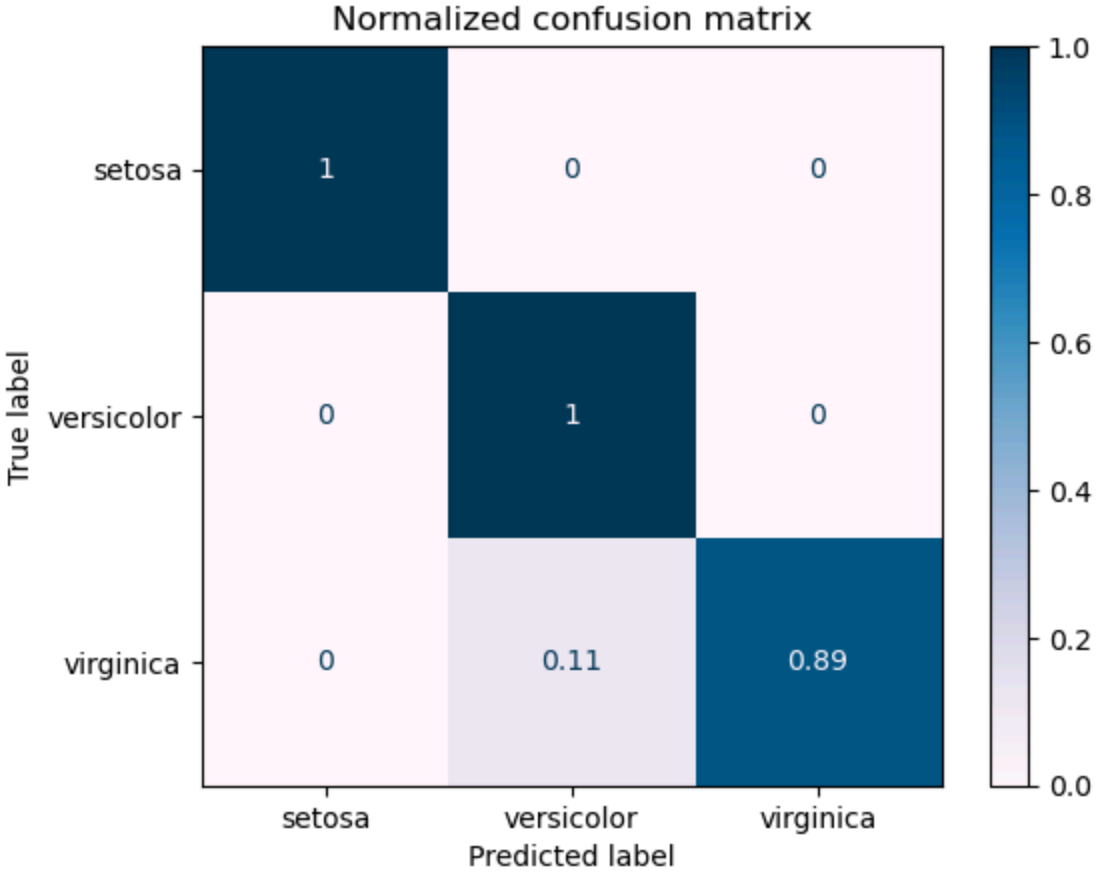
titles_options = [
    ("Confusion matrix, without normalization", None),
    ("Normalized confusion matrix", "true"),
]
for title, normalize in titles_options:
    disp = ConfusionMatrixDisplay.from_estimator(
        tree_clf,
        fitted_test_x,
        test_y,
        display_labels=labels,
        cmap=plt.cm.PuBu,
        normalize=normalize,
    )
    disp.ax_.set_title(title)

    print(title)
    print(disp.confusion_matrix)

plt.show()
```

```
Confusion matrix, without normalization
[[17  0  0]
 [ 0 10  0]
 [ 0  2 16]]
Normalized confusion matrix
[[1.   0.   0.  ]
 [0.   1.   0.  ]
 [0.   0.11 0.89]]
```


Confusion matrix, without normalization
setosa
versicolor
virginica
True label
17
0
0
0
10
0
0
2
16
setosa
versicolor
virginica
Predicted label
16
14
12
10
8
6
4
2
0


Normalized confusion matrix
setosa
versicolor
virginica
True label
1
0
0
0
1
0
0
0.11
0.89
setosa
versicolor
virginica
Predicted label
1.0
0.8
0.6
0.4
0.2
0.0

2.3 Thử điều chỉnh các tham số của cây quyết định và chọn tham số cho kết quả là giá trị f-score tốt nhất

- Thử nhiều tham số cho mô hình test từng các thông số thông qua `GridSearchCV`

In [30]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

params = {
    'max_depth': [2, 3, 4, 5, 6, None],
    'min_samples_split': [2, 3, 4, 5, 10],
    'min_samples_leaf': [1, 2, 3, 4, 5],
    'criterion': ['gini', 'entropy'],
}

grid = GridSearchCV(DecisionTreeClassifier(random_state=42), params, cv=5, s
grid.fit(fitted_train_x, train_y)
print(grid.best_params_)
best_tree = grid.best_estimator_
```

```
{'criterion': 'gini', 'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_sp
lit': 2}
```

Kết quả:

- sử thang đo `gini`
- độ xâu của cây là `5`
- số lá it nhất là `1`

In [31]:
```
best_pred = best_tree.predict(fitted_test_x)
```

In [32]:
```
print(classification_report(test_y, pred))
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        17
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.89      0.94        18

    accuracy                           0.96        45
   macro avg       0.94      0.96      0.95        45
weighted avg       0.96      0.96      0.96        45
```

In [ ]:
```

```